

Implementation Report

U-Net: Convolutional Networks for Biomedical Image Segmentation
A Faithful PyTorch Reimplementation of Ronneberger, Fischer & Brox (2015)

Deliverable: **UNet_Biomedical_Segmentation.ipynb** (Google Colab notebook)

Framework: PyTorch | Environment: Google Colab (GPU)

Reference paper: Ronneberger, O., Fischer, P., Brox, T. (2015). U-Net: Convolutional Networks for Biomedical Image Segmentation. MICCAI 2015, LNCS 9351, pp. 234–241.

This report documents the design, architecture, training methodology, and evaluation framework implemented in the accompanying notebook. It is a technical description of the implementation as delivered; the notebook has not been executed in the environment producing this report, so no trained-model performance numbers are quoted here — running the notebook's training cell populates the metrics and figures described in Sections 8–10.

1. Executive Summary

This report describes a standalone, GPU-ready PyTorch notebook that reimplements the U-Net architecture and training methodology introduced by Ronneberger, Fischer, and Brox (2015) for biomedical image segmentation. The notebook was built to reproduce every mechanism the paper identifies as central to its results — the unpadded convolutional architecture with cropped skip connections, He/Kaiming weight initialization, the pixel-wise weighted cross-entropy loss with its instance-separation weight map, elastic-deformation augmentation, and high-momentum SGD optimization — while remaining runnable end-to-end without any external dataset upload.

Because biomedical datasets used in the original paper (ISBI EM segmentation, PhC-U373, DIC-HeLa) are either gated behind challenge registration or require manual download, the notebook instead generates a synthetic “touching-cells” dataset that reproduces the two properties the paper’s methodology specifically targets: very small training-set size and densely touching object instances. The implementation is dataset-agnostic by design: swapping in a real dataset (Kaggle 2018 Data Science Bowl, ISBI EM, Oxford-IIIT Pet) requires replacing a single class.

Key deliverable characteristics

- 34 notebook cells (17 markdown, 17 code), organized into 9 numbered sections mirroring the paper’s structure.
- Two model variants: **UNetOriginal** (paper-exact unpadded convolutions, 572→388-style) and **UNetPadded** (same-padding convenience variant, used by default at 256×256).
- Exact reproduction of Eq. (1) (weighted cross-entropy) and Eq. (2) (instance-separation weight map) from the paper.
- He/Kaiming initialization with an inline sanity check reproducing the paper’s own worked example ($N = 9 \cdot 64 = 576$).
- Elastic-deformation + geometric augmentation pipeline built on Albumentations.
- SGD optimizer configured with momentum = 0.99 as specified in the paper’s training section.
- IoU (Jaccard) and pixel-accuracy metrics, training-curve plots, and 4-panel qualitative visualizations (image / ground truth / weight map / prediction).

2. Objective & Scope

The objective of this implementation was to produce a single, self-contained Colab notebook that: (a) reproduces the U-Net architecture and training recipe as specified in the original paper with high fidelity, (b) is immediately executable without manual dataset preparation, and (c) exposes every paper-specific mechanism (weight maps, elastic deformation, high-momentum optimization) as clearly separated, inspectable code rather than a black-box training script.

In scope

- Model architecture (both unpadded-original and same-padding variants).
- Weighted loss function and its associated weight-map preprocessing utility.

- Data augmentation pipeline (elastic deformation, flips, rotation, shift, intensity jitter).
- Training loop with checkpointing, learning-rate scheduling, and metric logging.
- Evaluation metrics (IoU, pixel accuracy) and qualitative visualization utilities.
- A synthetic dataset generator that stands in for real microscopy data.

Out of scope

- Training on the actual ISBI EM / PhC-U373 / DIC-HeLa datasets (these require external download/registration; the notebook documents how to substitute them in Section 9 of the notebook).
- Multi-GPU / distributed training.
- Hyperparameter search or benchmarking against other segmentation architectures.
- Deployment or inference-serving concerns.

3. Reference Paper Summary

Ronneberger et al. (2015) propose a fully convolutional architecture for biomedical image segmentation designed to work well with very few annotated training images. The network has a u-shaped structure: a contracting path that follows a typical convolutional-network pattern (repeated 3×3 unpadded convolutions and ReLU, 2×2 max-pooling with stride 2, doubling feature channels at each downsampling step) and a symmetric expansive path that upsamples the feature map, halves the channel count via a 2×2 “up-convolution,” concatenates the corresponding — cropped — feature map from the contracting path, and applies two further 3×3 convolutions. The network has no fully connected layers and consists of 23 convolutional layers in total. Because only “valid” convolutions are used, the output segmentation map is smaller than the input (572×572 in, 388×388 out in the paper's reference configuration), which the authors exploit via an overlap-tile strategy for seamless segmentation of arbitrarily large images.

Two additional ideas distinguish the paper's training recipe from a generic segmentation setup: (1) aggressive elastic-deformation data augmentation, motivated by the observation that deformation is the dominant source of realistic variation in biological tissue, and (2) a pixel-wise weighted cross-entropy loss (Eq. 1) whose weight map (Eq. 2) is precomputed per ground-truth mask to up-weight the thin background borders separating touching cell instances — directly addressing a failure mode (merged instances) that a plain cross-entropy loss does not penalize enough. Trained with SGD at momentum 0.99 and batch size 1, the network won the ISBI 2012 EM segmentation challenge and the ISBI 2015 cell-tracking challenge by a large margin, with inference on a 512×512 image taking under one second on a contemporary GPU.

4. Dataset: Synthetic Touching-Cell Generator

The notebook uses a purpose-built synthetic dataset in place of the paper's proprietary/gated datasets, so that the notebook runs standalone with no upload step. `generate_synthetic_sample()` renders a configurable number of irregular, ellipse-like blobs (approximating cell contours via jittered polygon vertices) onto a textured, noisy background using OpenCV, producing an **(image, binary mask)** pair per call. `SyntheticCellDataset` wraps a fixed, seeded pool of these samples into a standard `torch.utils.data.Dataset`.

Design rationale

- **Small sample counts by default** (40 train / 10 validation) — deliberately echoing the paper's own regime of 20–35 annotated images per task, to keep the augmentation and weighted-loss mechanisms meaningfully exercised rather than masked by data abundance.
- **Deliberately touching/overlapping instances** — blob centers and radii are sampled with enough density that neighboring cells frequently abut, which is the exact scenario Eq. (2)'s boundary term is designed to penalize (paper Fig. 3).
- **Seeded generation** — every sample is generated from an explicit integer seed, making the dataset reproducible across runs and separating the train/val pools by seed offset (0 vs. 10,000) to guarantee no overlap.
- **Drop-in replaceability** — because the rest of the pipeline (weight maps, augmentation, loss, training loop) only depends on the **(image[H,W] float32, mask[H,W] int64)** contract, Section 9 of the notebook documents substituting the Kaggle 2018 Data Science Bowl nuclei dataset, the ISBI EM challenge data, or **torchvision.datasets.OxfordIIITPet** without touching any other component.

5. Model Architecture

Both architecture variants share the same building block, **DoubleConv** — two 3×3 convolutions each followed by ReLU — and the same four-stage encoder/decoder topology with 2×2 max-pooling for downsampling and 2×2 transposed convolutions (“up-convolutions”) for upsampling, exactly mirroring Fig. 1 of the paper.

5.1 UNetOriginal — paper-exact, unpadded convolutions

Every convolution uses **padding=0** (“valid” convolution), so spatial resolution shrinks at every layer, matching the paper precisely. Skip-connection feature maps from the contracting path are larger than their corresponding decoder feature maps and must be spatially aligned before concatenation; this is handled by a dedicated **center_crop()** utility applied at each of the four decoder stages, directly implementing the paper's statement that “the cropping is necessary due to the loss of border pixels in every convolution.” With the canonical 572×572 input the network reduces to a 388×388 output, matching the paper's reference figures; the notebook verifies the actual output size for whatever input resolution is configured via a dry-run forward pass rather than hard-coding it.

5.2 UNetPadded — same-padding convenience variant

Identical topology and layer count to UNetOriginal, but every convolution uses **padding=1**, so input and output spatial resolutions match exactly (default: 256×256 → 256×256 in this notebook; 512×512 → 512×512 is also supported by changing **IMAGE_SIZE**). This removes the need to crop targets/weight maps to a shrunken output size, simplifying data loading and downstream metric computation. This is the variant selected by default (**UNET_VARIANT = "padded"**) for this notebook's 256×256 synthetic images; switching to **"original"** activates UNetOriginal instead.

5.3 Weight initialization

weights_init_he() applies Kaiming-normal initialization (fan-in mode, ReLU nonlinearity) to every **Conv2d** and **ConvTranspose2d** layer, implementing the paper's prescription of drawing initial weights from a Gaussian with standard deviation $\sqrt{(2/N)}$, where N is the number of incoming nodes per neuron. The notebook includes an executable sanity check that reproduces the paper's own worked example — a 3×3 convolution with 64 input channels, giving $N = 3 \times 3 \times 64 = 576$ — and prints the resulting empirical standard deviation of the initialized weights alongside the theoretical target for direct comparison.

5.4 Architecture fidelity summary

Paper specification	Implementation
4 downsampling stages, doubling channels each stage	Implemented (64→128→256→512→1024 by default)
4 upsampling stages via 2×2 up-convolution	Implemented via <code>nn.ConvTranspose2d(kernel=2, stride=2)</code>
Skip connections with center-cropped concatenation	Implemented (UNetOriginal only; UNetPadded needs no crop)
No fully-connected layers	Confirmed — final 1×1 Conv2d maps to <code>n_classes</code>
23 convolutional layers total	Reproduced identically in both variants (verified at runtime by counting Conv2d/ConvTranspose2d modules)
He/Kaiming initialization, $\text{std} = \sqrt{(2/N)}$	Implemented via <code>nn.init.kaiming_normal_, fan_in</code> mode
572×572 → 388×388 (reference config)	Reproducible via UNetOriginal; actual output size computed by dry-run rather than assumed

6. Weighted Loss Function

6.1 Weight map — Eq. (2)

$$w(x) = w_c(x) + w_0 * \exp(-((d1(x) + d2(x))^2) / (2 * \sigma^2))$$

compute_weight_map() composes two components exactly as specified:

- **w_c(x)** — a class-balancing term computed from inverse pixel-frequency of foreground vs. background in the ground-truth mask (**compute_class_balance_weight()**), normalized so the mean weight over the image is approximately 1.
- **Boundary term** — computed by **compute_boundary_weight()** using `scipy.ndimage.label` to obtain per-instance connected components (since the paper's mask is instance-aware even though the network's output is not), then `scipy.ndimage.distance_transform_edt` to compute, for every pixel, the Euclidean distance to each instance's boundary. The two smallest per-pixel distances (d_1, d_2 — nearest and second-nearest instance) are combined via the paper's Gaussian falloff with the paper's default constants **w₀ = 10.0** and **sigma = 5.0**, and the term is masked to apply only to background pixels, matching the paper's description of penalizing “the separating background labels between touching cells.”

6.2 Weighted cross-entropy — Eq. (1)

$$E = \sum_x w(x) * \log(p_l(x)(x))$$

WeightedCrossEntropyLoss computes a per-pixel log-softmax over the network's class logits, applies **F.nll_loss** with **reduction="none"** to obtain an unreduced per-pixel negative log-likelihood map, multiplies it elementwise by the precomputed weight map, and normalizes by the sum of weights. Weight maps are computed once per sample inside the dataset wrapper (**UNetSegmentationDataset**) — after augmentation, so that the boundary term reflects the post-augmentation instance geometry — and passed through the **DataLoader** alongside the image and mask tensors.

7. Data Augmentation

Section 3.1 of the paper identifies random elastic deformation, sampled on a coarse 3×3 grid with displacements drawn from a Gaussian of 10-pixel standard deviation and applied via bicubic interpolation, as the key augmentation enabling training from very few annotated images, alongside shift/rotation invariance and gray-value robustness. The notebook's **get_train_augmentation()** builds an Albumentations pipeline reproducing this recipe:

Transform	Purpose / paper correspondence
ShiftScaleRotate (shift 0.08, scale 0.1, rotate 25°, p=0.8)	Shift and rotation invariance
HorizontalFlip / VerticalFlip (p=0.5 each)	Additional geometric invariance
ElasticTransform (alpha=34, sigma=10, alpha_affine=6, p=0.8)	Paper §3.1's coarse, smooth elastic deformation field (10px-scale displacement)
RandomBrightnessContrast (±0.15, p=0.5)	Gray-value robustness
GaussNoise (var 0.0005–0.003, p=0.3)	Sensor-noise / imaging-condition robustness

Validation data uses an identity augmentation (**get_val_augmentation()**) so that reported metrics reflect un-augmented performance. Because the weight map depends on instance geometry, it is deliberately recomputed *after* augmentation for every sample rather than augmented as a pre-existing array, avoiding interpolation artifacts in the weight values themselves.

8. Training Configuration

Hyperparameter	Value	Paper correspondence
Optimizer	SGD	Matches paper (Caffe SGD implementation)
Momentum	0.99	Explicit paper value ("high momentum", §3)
Learning rate	1e-3 (initial), ReduceLROnPlateau (factor 0.5, patience 4, monitored on val IoU)	Not specified numerically in the paper; scheduler added as a practical stabilizer

Hyperparameter	Value	Paper correspondence
Batch size	2 (configurable; paper uses 1)	Paper favors large input tiles over batch size due to GPU memory limits; kept configurable since GPUs vary
Epochs	25 (configurable)	Not paper-specified; sized for the small synthetic dataset
Loss	WeightedCrossEntropyLoss (Eq. 1)	Exact
Checkpointing	Best validation IoU, saved with optimizer state + epoch + variant	Practical addition

The training loop (**train_one_epoch** / **validate**) iterates the weighted loss over (**image**, **mask**, **weight**) triples, backpropagates, and steps the optimizer per batch; validation additionally computes IoU and pixel accuracy per batch and averages them across the validation set. The best-performing checkpoint by validation IoU is persisted to **UNET_best.pt** and reloaded for the qualitative visualization stage, so the figures produced in Section 8 of the notebook always reflect the best observed model rather than the final epoch.

9. Evaluation Metrics

Two metrics are implemented, both computed with `@torch.no_grad()` for efficiency:

- **Mean IoU / Jaccard index (`compute_iou`)** — computed per class (background, foreground) as intersection over union between predicted and ground-truth pixel sets, then averaged across classes present in the batch. This is the metric Table 2 of the paper reports for the PhC-U373 and DIC-HeLa challenges (92.03% and 77.56% respectively for the paper's trained model).
- **Pixel accuracy (`compute_pixel_accuracy`)** — the fraction of pixels whose predicted class matches the ground truth, computed globally over the image.

Both metrics are logged per epoch into a **history** dictionary alongside train/validation loss, and plotted at the end of training (loss curves, validation IoU curve, validation pixel-accuracy curve). A final evaluation pass over the full validation set reports mean $\pm$ standard deviation for both metrics.

10. Execution Status & Expected Outputs

This notebook has not been executed in the environment that produced this report — all code cells carry empty **outputs** arrays and **execution_count: null** in the underlying .ipynb JSON. No training-run metrics, loss values, or IoU scores are therefore reported here, to avoid presenting fabricated results. Running the notebook top-to-bottom in a GPU-enabled Colab runtime will populate the following, in order:

- A printed PyTorch/CUDA/GPU environment summary (Section 0).
- A 2×3 grid of raw synthetic image/mask pairs (Section 1.2).
- A 3-panel figure showing an input image, its ground-truth mask, and its Eq. (2) weight map on a densely touching-cell sample (Section 2.2) — the weight map should visibly highlight thin bright lines along the background gaps between adjacent cells.

- A per-epoch training log line (train loss, validation loss, validation IoU, validation pixel accuracy, current learning rate) for each of the 25 configured epochs (Section 7.4), followed by a 3-panel training-curve figure (Section 7.5).
- A 4-row $\times$ 4-column qualitative grid (input / ground truth / weight map / prediction) for four validation samples using the best checkpoint (Section 8.1), and a final mean $\pm$ std IoU / pixel-accuracy summary over the full validation set (Section 8.2).

On the default synthetic dataset (40 training / 10 validation images at 256 $\times$ 256, 25 epochs, UNetPadded variant), the pipeline is expected to converge to a visibly coherent segmentation of the blob-like synthetic cells; because the synthetic task is considerably simpler and more consistent than real microscopy data, resulting IoU values should not be interpreted as representative of performance on the paper's actual EM/PhC-U373/DIC-HeLa benchmarks.

11. Fidelity to the Original Paper — Full Cross-Reference

Paper element	Notebook section / object	Fidelity
Fig. 1 architecture (unpadded convs, 572 $\rightarrow$ 388)	Sec. 4 — UNetOriginal	Exact
Convenience same-padding variant	Sec. 4 — UNetPadded	Extension (not in paper)
He/Kaiming init, N=9-64 worked example	Sec. 4.4 — <code>weights_init_he()</code>	Exact, with inline verification
Eq. (2) weight map (<code>w_c</code> , <code>d1</code> , <code>d2</code> , <code>w0=10</code> , $\sigma=5$)	Sec. 3 — <code>compute_weight_map()</code>	Exact
Eq. (1) weighted cross-entropy	Sec. 5 — <code>WeightedCrossEntropyLoss</code>	Exact
§3.1 elastic deformation, 3 $\times$ 3 grid, $\sigma=10$ px	Sec. 4 — <code>get_train_augmentation()</code>	Faithful approximation via Albumentations <code>ElasticTransform</code>
SGD, momentum 0.99, batch size 1	Sec. 7 — optimizer config	Momentum exact; batch size configurable (default 2) for practicality
Table 1/2 IoU / error metrics	Sec. 6 — <code>compute_iou</code> , <code>compute_pixel_accuracy</code>	IoU exact; paper's warping/Rand error metrics (EM-challenge-specific) not implemented
Fig. 2/3/4 qualitative figures	Sec. 8 — visualization cells	Reproduced in spirit (image /mask/weight-map/prediction panels)
30/35/20-image training sets (very few images)	Sec. 1 — <code>SyntheticCellDataset</code>	Analogous scale (40 train / 10 val) on synthetic data

Paper element	Notebook section / object	Fidelity
Overlap-tile seamless-tiling strategy (Fig. 2)	—	Not implemented (relevant only to UNetOriginal on arbitrarily large images; out of scope)
Warping error / Rand error (EM-challenge-specific)	—	Not implemented (challenge-specific metrics requiring the official evaluation server)

12. Limitations & Assumptions

- **Synthetic, not real, imaging data.** The blob-based generator captures touching-instance geometry but not the textures, staining artifacts, or acquisition noise of real EM/DIC/phase-contrast microscopy; quantitative results on it are not comparable to the paper's Table 1/2 figures.
- **Overlap-tile inference strategy not implemented.** The paper's approach to seamlessly segmenting arbitrarily large images by mirrored-border tiling (Fig. 2) is out of scope; the notebook performs single-pass inference at a fixed resolution.
- **Challenge-specific metrics omitted.** Warping error and Rand error (used for the ISBI EM leaderboard in Table 1) require the official challenge evaluation server and are not reproduced; IoU and pixel accuracy are used instead, matching Table 2's evaluation style.
- **Batch size defaults to 2, not 1.** The paper's batch size of 1 (chosen to maximize input tile size under GPU memory constraints) is preserved as a supported, documented configuration but not the notebook's default, since Colab GPU memory varies by session.
- **Elastic deformation parameters are an approximation.** Alumentations' ElasticTransform does not expose an explicit "3×3 control-grid" parameter as described in the paper; alpha/sigma were tuned to produce comparably smooth, coarse deformation fields rather than matching the paper's grid implementation bit-for-bit.
- **Not yet executed.** As noted in Section 10, no run has been performed against this deliverable; all described outputs are the notebook's designed behavior, not observed results.

13. How to Run & Extend

13.1 Running as-is

- Open **UNet_Biomedical_Segmentation.ipynb** in Google Colab.
- Set Runtime → Change runtime type → GPU (T4 or better).
- Run all cells top to bottom; Section 0.1 installs any missing dependencies (alumentations, opencv-python-headless, scipy) automatically.
- Training runs for 25 epochs by default (~minutes on a T4 given the small synthetic dataset and 256×256 resolution); adjust **NUM_EPOCHS**, **BATCH_SIZE**, or **IMAGE_SIZE** in Section 7.1/7.2 as needed.

13.2 Switching to the paper-exact unpadded architecture

Set `UNET_VARIANT = "original"` in Section 4.5. The notebook will dry-run the model to determine the true output resolution for the configured input size (rather than assuming 388 for a 572 input) and automatically propagate that target size through dataset cropping and loss computation.

13.3 Substituting a real biomedical dataset

Replace `SyntheticCellDataset` with a `torch.utils.data.Dataset` that returns `(image[H,W] float32 in [0,1], mask[H,W] int64)`. Section 9 of the notebook documents three concrete options:

- **Kaggle 2018 Data Science Bowl** (nuclei segmentation) — closest in spirit to the paper's touching-cell problem; requires a Kaggle API token uploaded to Colab.
- **ISBI EM segmentation challenge** — the exact dataset the paper reports its best score on (http://brainiac2.mit.edu/isbi_challenge/).
- **Oxford-IIIT Pet** (via `torchvision.datasets.OxfordIIITPet(..., target_types="segmentation")`) — a convenient sanity check on natural images without any manual download.

No other component (augmentation, weight-map computation, loss, model, training loop, metrics, visualization) requires modification when the dataset is swapped, provided the `(image, mask)` contract above is honored.

14. Conclusion

The delivered notebook implements the U-Net architecture and its paper-specific training methodology — unpadded convolutions with cropped skip connections, He initialization, the Eq. (2) instance-separation weight map, Eq. (1) weighted cross-entropy loss, elastic-deformation augmentation, and high-momentum SGD — as a single, runnable, dependency-light Colab notebook. A synthetic touching-cell dataset allows the full pipeline, including the paper's most distinctive mechanism (the boundary-aware weight map), to be exercised without any external data dependency, while the dataset abstraction is deliberately thin so that a real biomedical dataset can be substituted with a single class replacement. Section 12 above lists the specific points (synthetic vs. real imagery, omitted overlap-tile inference and challenge-specific error metrics, default batch size) where this implementation diverges from an exact reproduction of the paper's experimental setup.

15. Reference

Ronneberger, O., Fischer, P., & Brox, T. (2015). U-Net: Convolutional Networks for Biomedical Image Segmentation. In N. Navab et al. (Eds.), *Medical Image Computing and Computer-Assisted Intervention (MICCAI 2015)*, Part III, LNCS 9351, pp. 234–241. Springer International Publishing. DOI: 10.1007/978-3-319-24574-4_28.